IÉSEG SCHOOL OF MANAGEMENT

FUNDAMENTALS OF DEEP LEARNING:

FINAL PROJECT

Studen Name:
Qiushuang LIU
Duc Manh NGUYEN
Minh Hoan TRAN
Instructors Name: Shemair Floyd WILLIAMS
Date of Submission: 01/04/2026

TABLE OF CONTENTS

LIST OF ABBREVIATIONS **1**

LIST OF FIGURES AND TABLES **2**

INTRODUCTION **4**

PART 1: FLICKR IMAGE CLASSIFICATION **5**

1.1. Load the dataset and process the images **5**

1.1.1. Data Loading ***5***

1.1.2. Exploratory Data Analysis (EDA) ***5***

1.1.3. Image Preprocessing and Generators ***6***

1.1.4. Train two different Multi-layer perceptron networks ***7***

1.1.5. MLP Comparison Results ***8***

1.2. Train a CNN with the structure of the diagram presented in the APPENDIX. **8**

1.2.1. Define the number of epochs, batch size, and other parameters for training ***10***

1.2.2. Compare the predictive performance on both the training and validation sets ***10***

1.2.3. Observation ***11***

1.3. Improvement and Optimization **11**

1.3.1. CNN Optimization Experiments ***11***

1.3.2. Data Augmentation ***13***

1.3.3. Interpretability Techniques ***14***

1.3.4. Transfer Learning ***15***

1.4. Conclusion **16**

PART 2: EXPLORING RNNS AND CNNS FOR A SEQUENCE PREDICTION TASK **17**

2.1. Introduction **17**

2.2. Dataset Selection and Justification **17**

2.2.1. Dataset Overview *18*

2.3. Task Definition and Sequence Construction **18**

2.4. Methodology **19**

2.4.1. Data Preprocessing *19*

2.4.2. Train / Validation / Test Split *19*

2.4.3. Model Architectures *19*

2.4.4. Hyperparameter Design *20*

2.5. Results **20**

2.5.1. Baseline LSTM and GRU Comparison *20*

2.5.2. Best CNN Results *21*

2.5.3. Attention Mechanism and Examples *21*

2.5.4. Final Model Comparison *22*

2.6. Discussion **23**

2.6.1. Why GRU Performed Best *23*

2.6.2. Why Attention Performed Worse *23*

2.6.3. CNN vs RNN Interpretation *23*

2.7. Limitations and Future Work **23**

2.8. Conclusion **24**

OVERALL CONCLUSION **25**

LIST OF ABBREVIATIONS

Abbreviation	Full meaning
AI	Artificial Intelligence
ANN	Artificial Neural Network
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
GAP	Global Average Pooling
GELU	Gaussian Error Linear Unit
Grad-CAM	Gradient-weighted Class Activation Mapping
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MinMax	Min-Max Normalization
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
ReLU	Rectified Linear Unit
ResNet50	Residual Network with 50 layers
RNN	Recurrent Neural Network
VGG	Visual Geometry Group (deep CNN architecture)

LIST OF FIGURES AND TABLES

TABLES

Table 1.1: Comparison of MLP 1 (baseline) and MLP 2 (with regularization) results 8

Table 2.1: Summary of hyperparameter configurations 20

Table 2.2: Best configurations for baseline RNN models 20

Table 2.3: Final model configurations and comparison 22

FIGURES

Figure 1.1: Code implementation of data loading 5

Figure 1.2: Class distribution of the Flickr8k dataset 5

Figure 1.3: Sample images per category from the Flickr8k dataset 6

Figure 1.4: Code implementation of pipeline configuration for image preprocessing 6

Figure 1.5: Code implementation of MLP 1 and MLP 2 architectures 7

Figure 1.6: MLP training and validation accuracy/loss curves over 10 epochs 8

Figure 1.7: CNN architecture overview 9

Figure 1.8: Code implementation of CNN layer structure 9

Figure 1.9: Code implementation of CNN training configuration 10

Figure 1.10: CNN accuracy and loss curves 11

Figure 1.11: Code implementation of Experiment 1 (2×2 Pooling + GAP) 11

Figure 1.12: Code implementation of Experiment 2 (Batch Normalization) 12

Figure 1.13: Code implementation of Experiment 3 (Deeper CNN + Dropout) 12

Figure 1.14: Validation loss comparison across the three CNN optimization experiments 13

Figure 1.15: Code implementation Data augmentation 13

Figure 1.16: Data Augmentation Evidence 14

Figure 1.17: Code implementation of Saliency maps 14

Figure 1.18: Code implementation of Grad-CAM 15

Figure 1.19: Sample image applied Saliency Map and Grad-CAM 15

Figure 1.20: ResNet50 accuracy and loss curves 16

Figure 2.1: Daily mean temperature of London (2015–2021) 18

Figure 2.2: Example of 'Many-to-One' Sequence Construction 19

Figure 2.3: Best GRU vs CNN vs actual values 21

Figure 2.4: Attention weights for a sample sequence 21

Figure 2.5: Baseline GRU vs GRU + Attention vs actual values 22

INTRODUCTION

Deep learning has rapidly emerged as one of the most transformative areas of artificial intelligence, enabling machines to learn complex representations directly from raw data. From image recognition and natural language processing to time series forecasting, deep neural networks have demonstrated remarkable capabilities across a wide range of domains. This report documents the work carried out as part of the final project for the Fundamentals of Deep Learning course at IÉSEG School of Management, in which we design, train, evaluate, and compare a series of deep learning architectures on two distinct tasks.

The project is divided into two main parts. Part 1 focuses on image classification using the Flickr8k dataset, a collection of photographs spanning five categories: animals, objects, people, scenes, and others. We begin by loading and exploring the data, before building and refining a series of models - starting with Multi-Layer Perceptrons (MLPs) and advancing to Convolutional Neural Networks (CNNs). We then apply optimization strategies including modified pooling schemes, batch normalization, deeper architectures with dropout regularization, and data augmentation. To ensure our best model is interpretable, we apply Saliency Maps and Grad-CAM visualizations. Finally, we leverage Transfer Learning with a pre-trained ResNet50 model to demonstrate the performance gains achievable by building on established feature extractors.

Part 2 shifts focus to sequential data and the problem of time series prediction. Using daily temperature recordings from London spanning multiple years, we construct a supervised learning task in which historical observations are used to forecast the following day's temperature. We implement and compare three families of models: Long Short-Term Memory networks (LSTMs), Gated Recurrent Units (GRUs), and one-dimensional CNNs. We additionally explore the effect of incorporating an attention mechanism on top of the best-performing RNN model, assessing whether enhanced interpretability translates into improved predictive accuracy. All models are evaluated systematically on a held-out test set using Mean Squared Error (MSE) and Mean Absolute Error (MAE) as primary metrics.

Throughout both parts, we place emphasis not only on predictive performance but also on understanding the trade-offs between model complexity, generalization, and computational efficiency. Each architectural decision is motivated by theoretical considerations and supported by empirical evidence drawn from training curves, validation metrics, and visual diagnostics.

We would like to express our sincere gratitude to Professor Shemair Floyd WILLIAMS for his guidance, expertise, and continuous support throughout this course. His clear instruction in both the theoretical foundations and practical applications of deep learning provided us with the tools and confidence to tackle the challenges presented in this project. We are grateful for the time and effort he dedicated to ensuring our understanding, and for his encouragement to think critically about our results.

PART 1: FLICKR IMAGE CLASSIFICATION

1.1. Load the dataset and process the images

We load the Flickr8k dataset, do EDA, and then preprocess the data before training the deep learning model. The purpose of this stage is to check the structure of the dataset, make the photos all the same size with the same pixel values.

1.1.1. Data Loading

We only load metadata from the train_image_class.csv and valid_image_class.csv files for pre-processing before training the model. This is to prevent processing vast numbers of photos directly in memory.

```
# 1.1 Implementation: Data Loading
import pandas as pd
import os

DATA_DIR = '../data/'
TRAIN_CSV = os.path.join(DATA_DIR, 'train_image_class.csv')
VALID_CSV = os.path.join(DATA_DIR, 'valid_image_class.csv')

train_df = pd.read_csv(TRAIN_CSV)
valid_df = pd.read_csv(VALID_CSV)
```

Figure 1.1: *Code implementation of data loading*

1.1.2. Exploratory Data Analysis (EDA)

We used countplots to see how the data was spread out and checked a few random image samples by category.

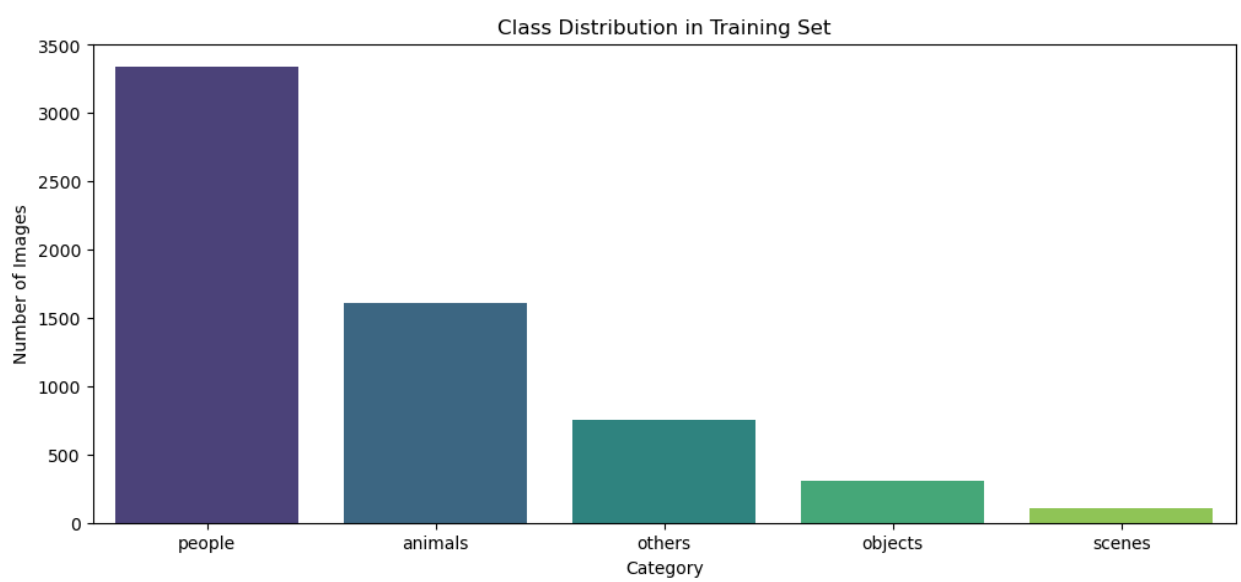

Figure 1.2: *Class distribution of the Flickr8k dataset*

Figure 1.3: *Sample images per category from the Flickr8k dataset*

1.1.3. Image Preprocessing and Generators

We use ImageDataGenerator to construct a pipeline that normalizes and standardizes the input image data using the following settings:

- **rescale=1./255:** Normalize the pixel values to the range [0, 1] to maintain the gradient steady during the whole procedure.
- **target_size=(224, 224):** Standardize the input image for neural networks.
- **batch_size=32:** This parameter is used to optimize memory usage while ensuring a stable gradient.
- **class_mode='categorical'**: Since we need to perform multi-class classification, the labels need to be one-hot encoded.

```python
# 1.3 Implementation: Preprocessing and Generators
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(rescale=1./255)
IMG_SIZE = (224, 224)
BATCH_SIZE = 32

train_generator = datagen.flow_from_dataframe(
    dataframe=train_df,
    directory=DATA_DIR,
    x_col="Image Path",
    y_col="Label",
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    shuffle=True
)

valid_generator = datagen.flow_from_dataframe(
    dataframe=valid_df,
    directory=DATA_DIR,
    x_col="Image Path",
    y_col="Label",
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    shuffle=False
```

Figure 1.4: *Code implementation of pipeline configuration for image preprocessing*

1.1.4. Train two different Multi-layer perceptron networks

Before feeding images into a standard MLP, the 2D pixel matrices (224x224x3) must be flattened into a single 1D array. We build two different MLP architectures to compare how depth and regularization affect performance on high-dimensional image data.

- **MLP 1 (Baseline):** A simple network made to set a baseline. There are two hidden layers (64 and 32 neurons).
- **MLP 2 (Deeper with Regularization):** A deeper network (128, 64, and 32 neurons) with more capacity to learn. Because MLPs are prone to overfitting on complex image data, we add a Dropout layer to limit reliance on any single node.

```
# Build MLP 1
mlp1 = Sequential()
mlp1.add(Flatten(input_shape=(224, 224, 3))
mlp1.add(Dense(64, activation='relu'))
mlp1.add(Dense(32, activation='relu'))
mlp1.add(Dense(5, activation='softmax'))

# Build MLP 2
mlp2 = Sequential()
mlp2.add(Flatten(input_shape=(224, 224, 3))
mlp2.add(Dense(128, activation='relu'))
mlp2.add(Dropout(0.3))
mlp2.add(Dense(64, activation='relu'))
mlp2.add(Dense(32, activation='relu'))
mlp2.add(Dense(5, activation='softmax'))
```

Figure 1.5: *Code implementation of MLP 1 and MLP 2 architectures*

Parameters & Settings

- **Flatten()**: Flatten the image tensors, from the structure (224, 224, 3) to a 1D vector with 150,528 features so that the data can pass through dense layers
- **Dense(units)**: The number of units corresponds to the number of neurons corresponding to each dense layer. Here, we only choose a sufficient number of neurons to distinguish details; too many would easily cause overfitting.
- **activation='relu'**: Rectified Linear Units (ReLUs) make the model nonlinear, easily distinguishing complex patterns. In particular, this activation function reduces the likelihood of "vanishing gradients".
- **Dropout(0.3)**: To prevent the model from memorizing data, we turned off 30% of the network during training.
- **activation='softmax'**: When applying the softmax function to the final activation layer, we obtain the output as the classification probabilities of the images according to categories.

1.1.5. MLP Comparison Results

Table 1.1: *Comparison of MLP 1 (baseline) and MLP 2 (with regularization) results*

Model	Parameters	Best Val. Accuracy	Observation
MLP 1	9,636,101	54.81%	Reached a peak of 54.81% at Epoch 8 but dropped significantly to 48.07% by Epoch 9. This shows high instability without regularization.
MLP 2	19,278,213	53.37%	Extremely stable performance. Maintained exactly 53.37% validation accuracy for all 10 epochs, proving that Dropout successfully managed the 19.2M parameters.

Even though MLP 1 had a slightly higher peak, we can see that both models struggle with the high dimensionality of the data, specifically image data. To improve this, we will work with a CNN network.

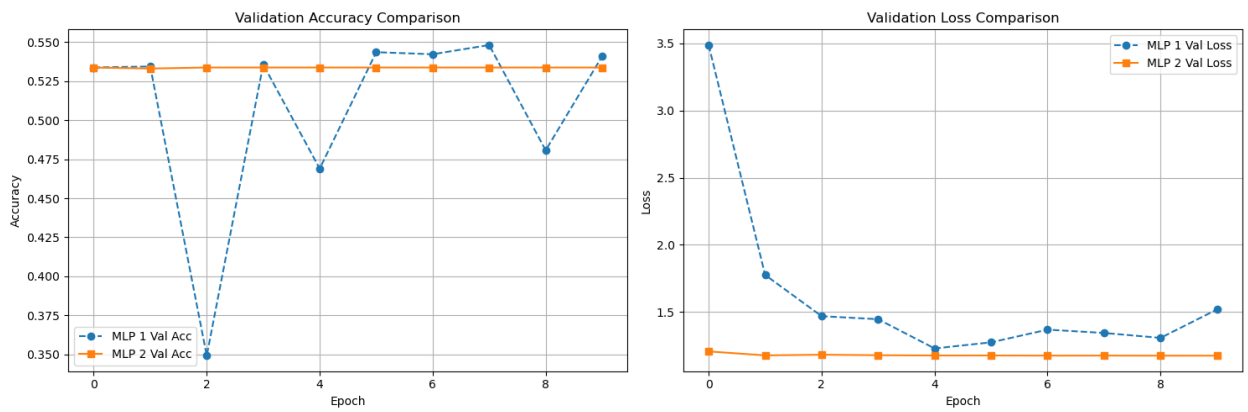

Figure 1.6: *MLP training and validation accuracy/loss curves over 10 epochs*

1.2. Train a CNN with the structure of the diagram presented in the APPENDIX.

We built a Convolutional Neural Network (CNN) based on the diagram our group was given. CNNs employ convolutional filters to find patterns and hierarchies in space, including edges and forms, right from 2D images. This means they are significantly better at sorting pictures than MLPs. There is a Feature Extraction step and subsequently a Classification Head in the architecture.

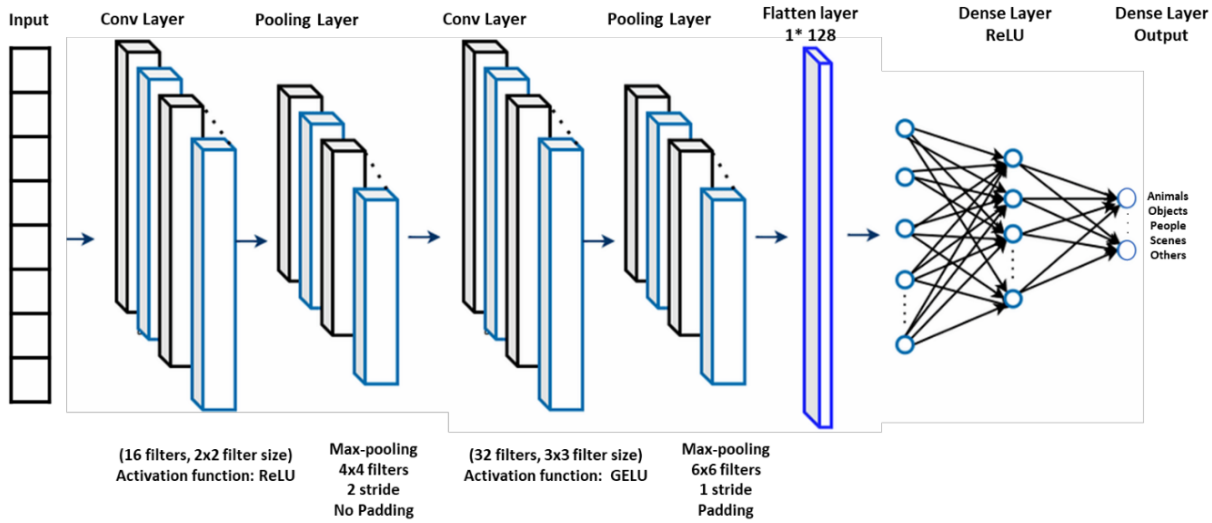

Figure 1.7: *CNN architecture overview*

```
model = Sequential()

# Conv Layer 1: 16 filters, 2x2, ReLU
model.add(Conv2D(16, (2, 2), activation='relu', input_shape=(224, 224, 3)))
# Max-Pooling 1: 4x4, 2 stride, No Padding
model.add(MaxPooling2D(pool_size=(4, 4), strides=(2, 2), padding='valid'))

# Conv Layer 2: 32 filters, 3x3, GELU (as specified in diagram)
model.add(Conv2D(32, (3, 3), activation='gelu'))
# Max-Pooling 2: 6x6, 1 stride, Padding
model.add(MaxPooling2D(pool_size=(6, 6), strides=(1, 1), padding='same'))

# Classification Head
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(5, activation='softmax'))
```

Figure 1.8: *Code implementation of CNN layer structure*

Feature Extraction Stage:

- **Conv2D(16, (2, 2), activation='relu'):** The first convolutional layer uses 16 filters, each of which is 2x2. The ReLU (Rectified Linear Unit) activation function is then applied. This makes the function non-linear by ignoring pixel values that are below zero.
- **MaxPooling2D((4, 4), strides=(2, 2), padding='valid'):** This pooling layer uses a 4x4 window and a stride of 2 to reduce the spatial dimensions without adding any padding. This approach decreases the computational load and makes the model easier to process.
- **Conv2D(32, (3, 3), activation='gelu'):** The second convolutional layer which increases the depth to include 32 filters, each measuring 3x3. This layer uses the GELU (Gaussian Error Linear Unit) activation function, which, in some cases, can enable smoother gradient flow compared to the ReLU function.

- **MaxPooling2D((6, 6), strides=(1, 1), padding='same'):** This second pooling layer uses a large 6x6 window but a stride of 1. Using 'same' padding means the output feature map will have the same spatial dimensions as the input to this layer.

Classification Head:

- **Flatten():** Change the multi-dimensional feature maps from the convolutional base into a one-dimensional vector.
- **Dense(128, activation='relu'):** A fully connected layer which has 128 neurons and uses the ReLU activation function. It processes the features that have been extracted, allowing for the making of more complex decisions.
- **Dense(5, activation='softmax'):** The last output layer has 5 neurons, which represent the five different classes: animals, objects, people, scenes, and others. The softmax function is used to turn the raw outputs into probabilities, making sure that their total value is 1.

1.2.1. Define the number of epochs, batch size, and other parameters for training

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
history = model.fit(train_generator, validation_data=valid_generator, epochs=15)
```

Figure 1.9: *Code implementation of CNN training configuration*

To ensure effective training of the CNN architecture, the following parameters were selected:

- **Optimizer='adam':** This optimizer was chosen because its adaptive learning rate properties are important for keeping training stable in architectures with a lot of parameters (47.7M) and a lot of lost spatial information.
- **Loss= 'categorical_crossentrophy':** This is the main evaluation metric because it gives us a direct and easy way to see how many of the 5 categories were correctly classified. This makes it easy for us to keep an eye on how well the model is doing during training.
- **Epochs=15:** The empirical study indicates that the model attains its peak validation accuracy early on, notably at Epoch 3. Fifteen epochs is enough to find the point at which the training accuracy and validation accuracy start to diverge.

1.2.2. Compare the predictive performance on both the training and validation sets

Even though the architecture was heavier (47.7M vs. 19.2M parameters), the CNN was more accurate than the MLP, reaching 59.32% accuracy at Epoch 3. But the large number of factors shows that we need to discover better ways to group them.

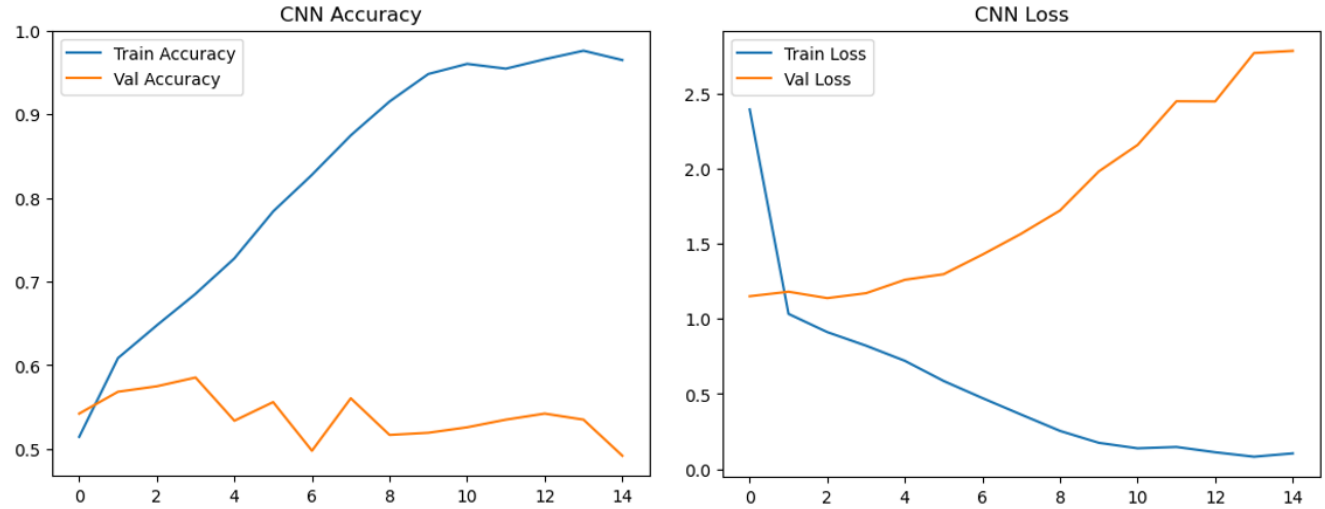

Figure 1.10: *CNN accuracy and loss curves*

1.2.3. Observation

- The model had higher accuracy than the MLP, but it also showed evidence of overfitting, with validation accuracy going down and loss going up in the later epochs.
- The validation curve's divergence tendency shows that big pooling windows, such 4x4 and 6x6, are too aggressive. This makes the model lose important spatial detail early on in the network.

1.3. Improvement and Optimization

1.3.1. CNN Optimization Experiments

There are three experiments to improve CNN based on the results we got before. Using the best practices learned in class, we changed the network settings to improve its performance.

- **Experiment 1 (Spatial Detail Retention):** The baseline model used 4x4 and 6x6 pooling windows, which destroyed the spatial details way too fast. To fix this, we used standard 2x2 MaxPooling2D layers. We also dropped the Flatten layer and used a GlobalAveragePooling2D (GAP) layer before the final Dense output. This cut down our parameter count heavily and kept overfitting in check.

```
exp1 = Sequential()
exp1.add(Conv2D(16, (3, 3), padding='same', activation='relu', input_shape=(224, 224, 3)))
exp1.add(MaxPooling2D(pool_size=(2, 2)))
exp1.add(Conv2D(32, (3, 3), padding='same', activation='relu'))
exp1.add(MaxPooling2D(pool_size=(2, 2)))
exp1.add(GlobalAveragePooling2D())
exp1.add(Dense(5, activation='softmax'))

exp1.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Figure 1.11: *Code implementation of Experiment 1 (2×2 Pooling + GAP)*

- **Experiment 2 (Batch Normalization):** We placed the BatchNormalization layers after each Conv2D layer, right before the ReLU activation functions to minimize the validation loss. This normalizes the input between layers and makes the gradient descent algorithm much more stable during training.

```
exp2 = Sequential()
exp2.add(Conv2D(16, (3, 3), padding='same', input_shape=(224, 224, 3)))
exp2.add(BatchNormalization())
exp2.add(Activation('relu'))
exp2.add(MaxPooling2D(pool_size=(2, 2)))

exp2.add(Conv2D(32, (3, 3), padding='same'))
exp2.add(BatchNormalization())
exp2.add(Activation('relu'))
exp2.add(MaxPooling2D(pool_size=(2, 2)))

exp2.add(Flatten())
exp2.add(Dense(128, activation='relu'))
exp2.add(Dense(5, activation='softmax'))

exp2.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Figure 1.12: *Code implementation of Experiment 2 (Batch Normalization)*

- **Experiment 3 (Deeper Architecture with Regularization):** We constructed a deeper network, VGG type, using stacked convolutional layers (two layers of 32 filters followed by two layers of 64 filters). To minimize overfitting, we used a 25% dropout rate after the pooling layers and a 50% dropout rate after the fully connected Dense(512) layer.

```
exp3 = Sequential()
exp3.add(Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=(224, 224, 3)))
exp3.add(Conv2D(32, (3, 3), activation='relu'))
exp3.add(MaxPooling2D(pool_size=(2, 2)))
exp3.add(Dropout(0.25))

exp3.add(Conv2D(64, (3, 3), padding='same', activation='relu'))
exp3.add(Conv2D(64, (3, 3), activation='relu'))
exp3.add(MaxPooling2D(pool_size=(2, 2)))
exp3.add(Dropout(0.25))

exp3.add(Flatten())
exp3.add(Dense(512, activation='relu'))
exp3.add(Dropout(0.5))
exp3.add(Dense(5, activation='softmax'))

exp3.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Figure 1.13: *Code implementation of Experiment 3 (Deeper CNN + Dropout)*

Result Analysis: Due to the validation loss chart, Experiment 1 (2x2 Pooling + GAP) is the best and most stable model. The loss line stays low and flat for all 10 epochs, which means the model didn't just memorize the training data. Experiment 2 (Batch Normalization) goes up and down a lot and the loss gets higher near the end. Experiment 3 (Deeper CNN + Dropout) is an example of overfitting. It starts well, but after Epoch 4, the loss passes 1.9. This result proves that using standard 2x2 pooling and Global Average Pooling layer was the best way to improve CNN.

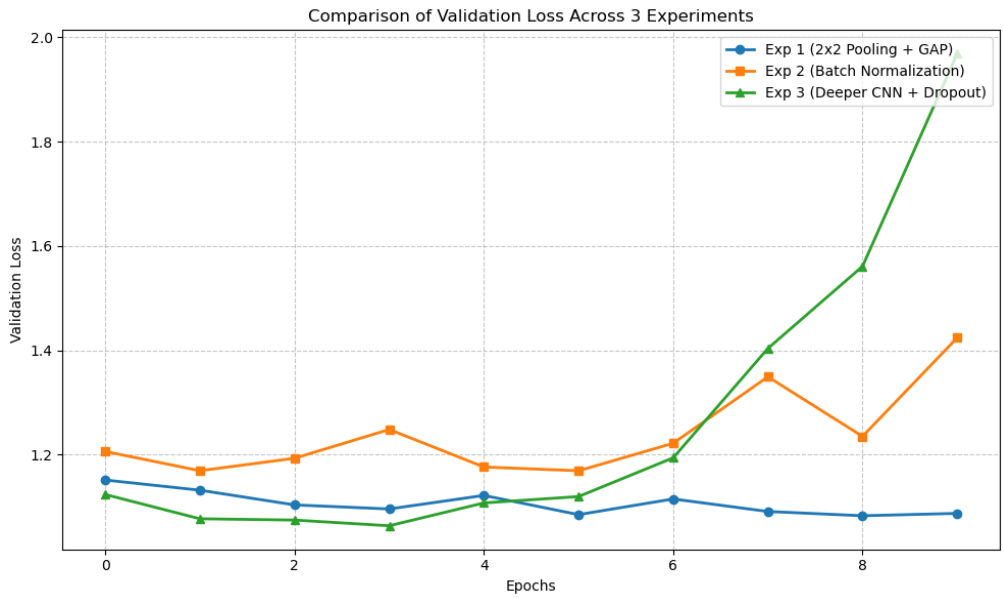

Figure 1.14: *Validation loss comparison across the three CNN optimization experiments*

1.3.2. Data Augmentation

To further improve our best model, we have applied data enhancement techniques.

- **Procedure:** Using Keras's "ImageDataGenerator," we applied transformations to the training dataset dynamically during each epoch. The parameters included a rotation range of 20 degrees, width and height shifts of 20%, shear and zoom ranges of 15%, and horizontal flipping.

```
# Create an augmented generator
train_datagen_aug = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.15,
    zoom_range=0.15,
    horizontal_flip=True,
    fill_mode='nearest'
)

train_generator_aug = train_datagen_aug.flow_from_dataframe(
    dataframe=train_df,
    directory=IMAGES_DIR,
    x_col='Image Path',
    y_col='Label',
    target_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=True
)
```

Figure 1.15: *Code implementation Data augmentation*

- **Results:** By exposing the model to these slightly altered versions of the images, we prevented the network from memorizing exact pixel locations. The retrained model demonstrated significantly

improved robustness, and the validation loss curve stopped spiking upward, proving that the model was no longer heavily overfitting.

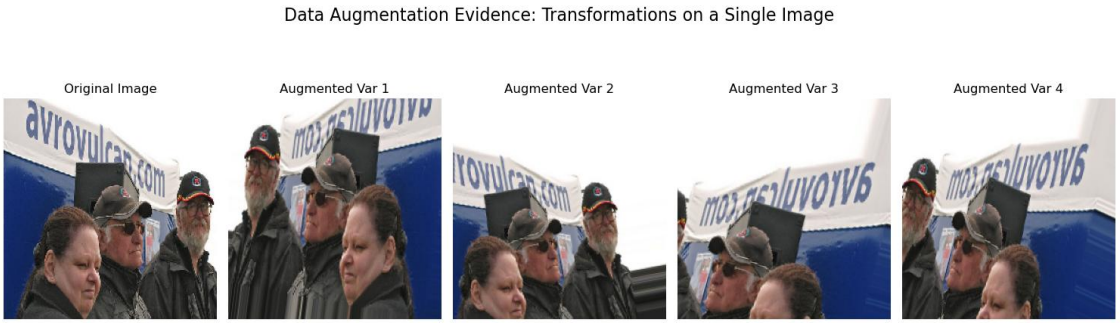

Figure 1.16: *Data Augmentation Evidence*

1.3.3. Interpretability Techniques

To ensure our CNN network makes predictions based on relevant image features rather than background details, we applied two techniques to explain our best-in-class augmented model.

- **Saliency Map (Vanilla Gradients):**The gradient of the highest predicted class score, relative to the input image pixels, was calculated. Subsequently, a map was produced by identifying the maximum absolute gradient across the color channels, thereby pinpointing the specific pixels that exerted the most significant localized influence on the model's classification.

```
# Technique 1: Saliency Map
def compute_saliency_map(model, img_array):
    img_tensor = tf.convert_to_tensor(img_array)
    with tf.GradientTape() as tape:
        tape.watch(img_tensor)
        predictions = model(img_tensor)[1]
        top_class = tf.argmax(predictions[0])
        top_class_pred = predictions[:, top_class]

    # Calculate the gradients of the prediction with respect to the input image
    gradients = tape.gradient(top_class_pred, img_tensor)
    saliency = tf.reduce_max(tf.abs(gradients), axis=-1)[0]
    return saliency.numpy()

saliency_map = compute_saliency_map(grad_model, sample_image_expanded)
```

Figure 1.17: *Code implementation of Saliency maps*

- **Grad-CAM (Gradient-weighted Class Activation Mapping):** We extracted the feature maps from the final convolutional layer of our model. Subsequently, we applied weights to these maps, using the pooled gradients associated with the predicted class. The resultant heatmap, when superimposed on the original image, offered a general localization of the overarching concepts that the network prioritized (for instance, the texture of a dog's coat or the form of an object).

```python
# Technique 2: Grad-CAM
def compute_gradcam(model, img_array):
    with tf.GradientTape() as tape:
        conv_outputs, predictions = model(img_array)
        top_class = tf.argmax(predictions[0])
        loss = predictions[:, top_class]

    # Calculate the gradients of the output with respect to the Conv layer
    grads = tape.gradient(loss, conv_outputs)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

    conv_outputs = conv_outputs[0]
    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = tf.maximum(heatmap, 0) / tf.math.reduce_max(heatmap)
    return heatmap.numpy()
```

Figure 1.18: *Code implementation of Grad-CAM*

Both visual explanations confirmed that the model focuses on the correct features (e.g., the face and body of the animal) rather than relying on background artifacts.

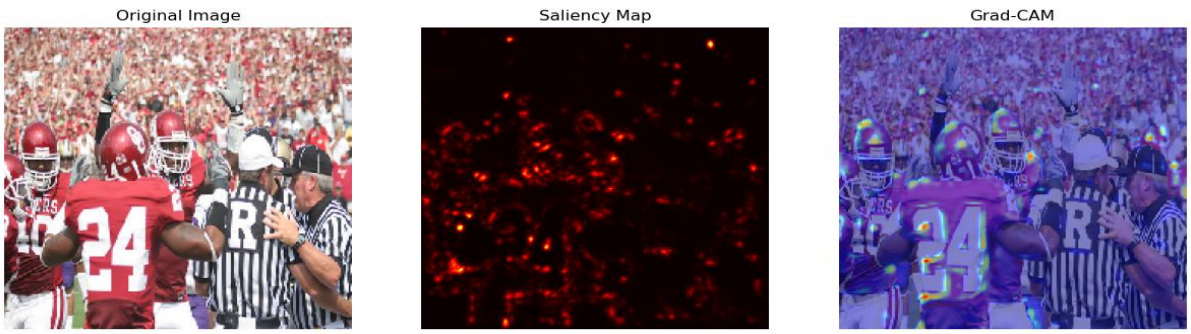

Figure 1.19: *Sample image applied Saliency Map and Grad-CAM*

1.3.4. Transfer Learning

In the project's final stage, we used Transfer Learning, specifically the ResNet50 architecture, directly from Keras.

- **Model Choice:** My team uses ResNet50 for learning transfer because it's a very good pretrained model on the ImageNet dataset, enabling it to extract image features effectively.
- **Modifications:** We adjusted several model parameters to make it more efficient with the dataset we received. Specifically, we used include_top=False to eliminate the ability to identify and classify 1000 different labels (classes), instead using only 5 classes for the flicker problem. We used trainable=False to preserve all the parameters of the ResNet50 model. We superimposed a GlobalAveragePooling2D layer, a Dense 512-node layer with the ReLU activation function, and a Dropout 0.5 layer for tuning. Finally, 5 nodes for softmax mapping were used to classify the 5 specific classes of the problem.

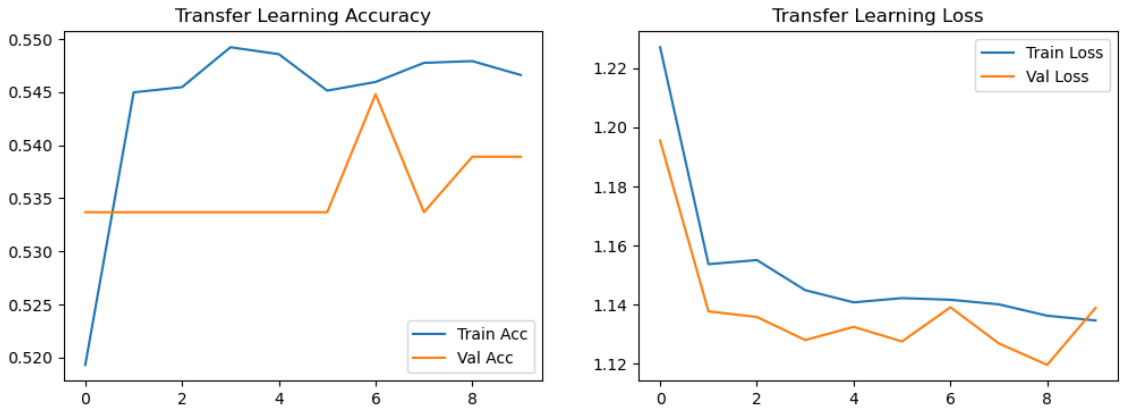

Figure 1.20: *ResNet50 accuracy and loss curves*

1.4. Conclusion

In summary, the overall performance of the models we have implemented is as follows:

- **MLP Models:** We started with a multilayer neural network (MLP), flattening 2D images into 1D and feeding them into a model. Our most complex MLP reached 19.2 million parameters, but its accuracy was still very low, only reaching about 53%, and it also required the use of many dropout techniques to function. This demonstrates that this type of MLP is not effective for the task of classifying and extracting features from image data.
- **Baseline CNN:** The convolutional filters we used helped the model recognize and extract features, increasing accuracy by 59%. However, this is still not a very good number, and our team also found that using 4x4 and 6x6 pooling windows made the model less capable of extracting image features in depth, leading to overfitting and very poor performance on unseen data sets.
- **Optimized CNN:** After three experiments, we found that using 2x2 pooling combined with the Global Average Pooling (GAP) layer was the best method to improve the model and avoid overfitting. In addition, combining this experiment with techniques such as data augmentation significantly improved model stability. This demonstrates that sometimes a simpler model structure is more efficient for certain problems and avoids the possibility of the model learning training data, leading to overfitting and poor performance in dealing with unseen data.
- **Transfer Learning (ResNet50):** ResNet50 is the best model after all the modifications our team made. The use of a pre-trained model significantly improved training and feature extraction capabilities compared to previous versions. This demonstrates that using a pre-trained model is the optimal method for this classification problem.

PART 2: EXPLORING RNNS AND CNNS FOR A SEQUENCE PREDICTION TASK

2.1. Introduction

This part of the project mainly focuses on the application of deep learning models in time series prediction. Unlike the image classification task in the first part, here we are concerned with how to use historical data to construct the optimal sequence model for predicting future values.

The aim of this study is to use the historical temperature data of London to predict the temperature for the next day. To achieve this, several deep learning architectures are compared, including Recurrent Neural Networks (LSTM and GRU), Convolutional Neural Networks (CNN), and an extended RNN model with an attention mechanism.

Accordingly, we have also thought about whether data pre-processing is equally important (e.g., data cleaning and normalization) so as to make the model learn faster and have better predictive power. In addition, we have assessed how well our model can generalize by splitting it into a training set, a validation set, and a test set. Thus, we make sure that our model performs reliably even on future unseen data. With such an approach, what we want to obtain is a more precise prediction on temperature.

2.2. Dataset Selection and Justification

Weather conditions have a significant impact in many practical applications, especially in F1 racing. Temperature directly affects the performance of tires, the conditions of the race track, and the performance of the drivers. Considering that London is geographically close to the famous Silverstone Circuit, this study selects the weather data of London as the basis for this prediction task.

The dataset used in this project is the London Weather dataset, available on Kaggle: https://www.kaggle.com/datasets/emmanuelfwerr/london-weather-data

This dataset contains daily meteorological observation data of the London, including temperature, precipitation, air pressure, and sunlight conditions.

In this study, we only retained the average temperature to construct a univariate time series.

This dataset is highly suitable for sequence prediction, and the reasons are as follows:

- Temperature exhibits strong temporal dependency
- The data includes both short-term fluctuations and seasonal patterns
- Future values depend on past observations
- The structure aligns well with RNN-based models

This task is a many-to-one sequence prediction problem, where a series of past values are used to predict the next value.

2.2.1. Dataset Overview

After preprocessing, only data from 2015 onward is retained to reduce noise and ensure relevance. This results in approximately 2,000+ observations, which is sufficient for training deep learning models.

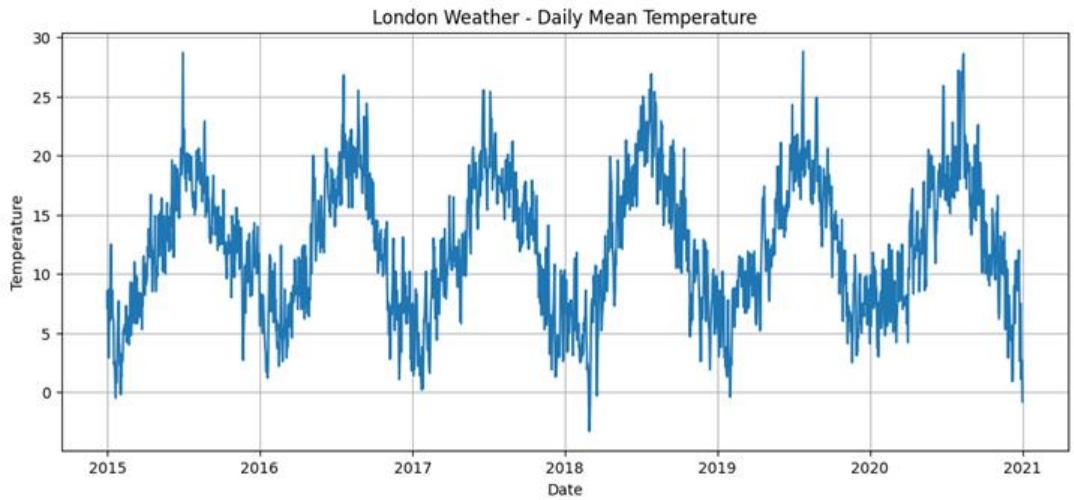

Figure 2.1: *Daily mean temperature of London (2015–2021)*

This figure illustrates the changing trend of London's temperature over time in this dataset, highlighting the seasonal patterns and smooth variations throughout the year. This indicates that this dataset is suitable for time series modeling.

2.3. Task Definition and Sequence Construction

To prepare the data for deep learning models, the time series is transformed into a supervised learning format.

Each sample consists of:

- **Input (X):** a sequence of past temperature values
- **Output (y):** the temperature of the next day

For example, using a sequence length of 7:

This formulation corresponds to a many-to-one prediction task and is highly suitable for models based on RNNs.

For illustration purposes, a sequence length of 7 is shown. However, the best-performing model uses a sequence length of 21.

Day1	Day2	Day3	Day4	Day5	Day6	Day7	→	Day8
7.1	8.6	4.9	2.9	4.9	8.7	6		5.6

Figure 2.2: *Example of 'Many-to-One' Sequence Construction*

2.4. Methodology

2.4.1. Data Preprocessing

The dataset is cleaned and prepared before training:

- Missing values are removed
- Data is sorted chronologically
- Only the temperature variable is retained

To improve model training stability, the data is normalized using MinMax scaling. This method will transform all values to the range of [0, 1]. The scaler was fitted only on the training data to avoid data leakage.

This ensures that the neural network can learn efficiently, thereby avoiding various problems arising from differences in data scales.

2.4.2. Train / Validation / Test Split

This dataset is split into the following parts in chronological order:

- Training set (70%)
- Validation set (10%)
- Test set (20%)

The validation set is used for model selection, while the test set is used only for final evaluation.

This method can effectively prevent data leakage and ensure fair comparisons among different models.

2.4.3. Model Architectures

Three types of models are implemented:

Recurrent Neural Networks (RNNs):

- **LSTM:** captures long-term dependencies
- **GRU:** a simplified version with fewer parameters

Both of these models process sequences step by step and retain temporal information.

Convolutional Neural Network (CNN):

Unlike directly simulating time dependence, convolutional neural networks extract local temporal patterns through convolutional filters. This enables convolutional neural networks to effectively capture the short-term relationships between adjacent time steps.

2.4.4. Hyperparameter Design

In order to ensure a systematic and fair comparison of these models, we tested multiple hyperparameters.

For the recurrent models, we systematically tested different hyperparameter configurations, including sequence length (3, 7, 12, 21), hidden units (32, 50, 64), and number of layers (1 or 2). For the CNN model, we tested different sequence lengths, numbers of filters, and kernel sizes. The best-performing baseline RNN was then extended with an attention mechanism, using the same sequence length and hidden units for a fair comparison.

Table 2.1: *Summary of hyperparameter configurations*

Model	Parameters Tested
LSTM	sequence length, units, layers
GRU	sequence length, units, layers
CNN	sequence length, filters, kernel size

We selected the best model based on validation loss and kept the test set only for the final evaluation.

2.5. Results

2.5.1. Baseline LSTM and GRU Comparison

After testing multiple configurations from different perspectives, we identified the best-performing versions of LSTM and GRU for this dataset.

Table 2.2: *Best configurations for baseline RNN models*

Model	Sequence_Length	Key_Params	Best_Val_Loss	Test_MSE	Test_MAE
Best GRU	21	units=32, layers=1	0.003293	4.843241	1.704301
Best LSTM	21	units=64, layers=1	0.003362	4.832592	1.714430

The results show that both models perform well, but GRU outperforms LSTM in this task.

2.5.2. Best CNN Results

In order to better evaluate the convolutional neural network (CNN) model, we also compared its prediction results with the actual values and the best-performing GRU model.

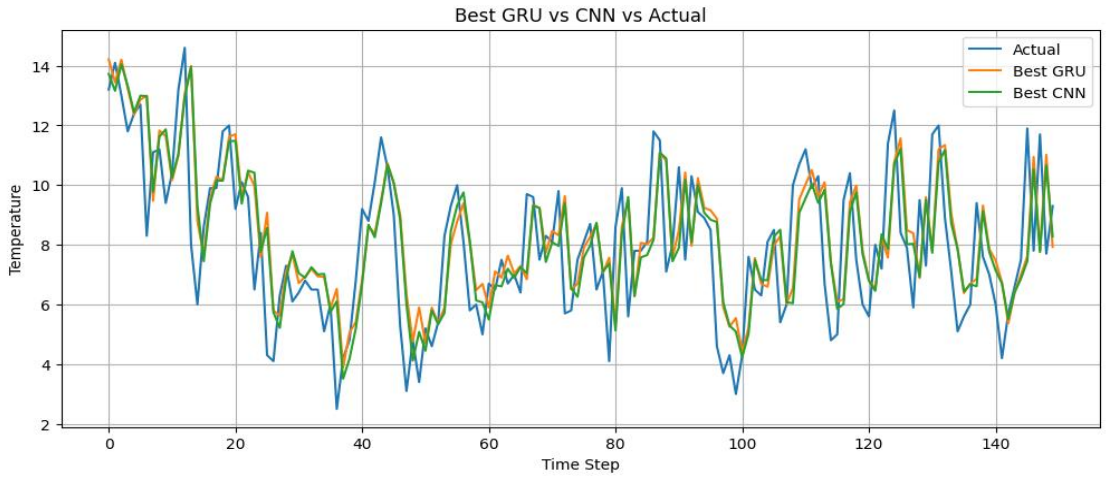

Figure 2.3: *Best GRU vs CNN vs actual values*

Figure 2.3 compares the predicted results and actual values of the best GRU model and CNN model. Although CNN effectively captures local patterns, its accuracy is slightly lower than that of the best RNN model we tested. In contrast, GRU produces predictions that are closer to the actual temperature trend.

2.5.3. Attention Mechanism and Examples

An attention mechanism was applied to the best-performing RNN model to enhance the interpretability of the model. The attention model was built using the same sequence length and hidden units as the best baseline model for a fair comparison.

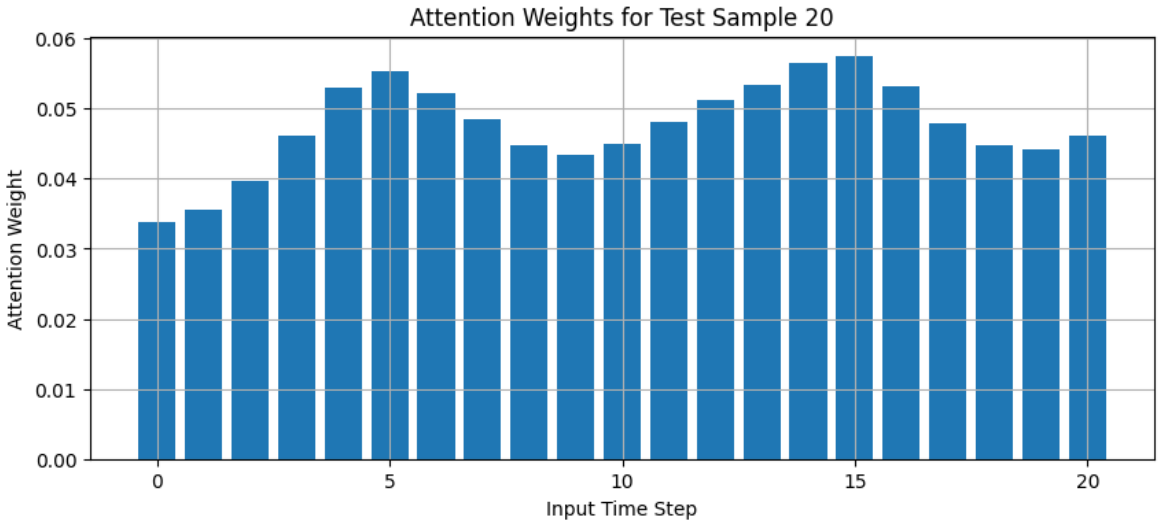

Figure 2.4: *Attention weights for a sample sequence*

Figure 2.4 illustrates the varying degrees of importance given to each time step by the model, indicating that certain past observations contribute more significantly to the prediction than others.

The attention weights show that the model assigns higher importance to certain time steps, especially more recent observations, indicating their stronger influence on the prediction.

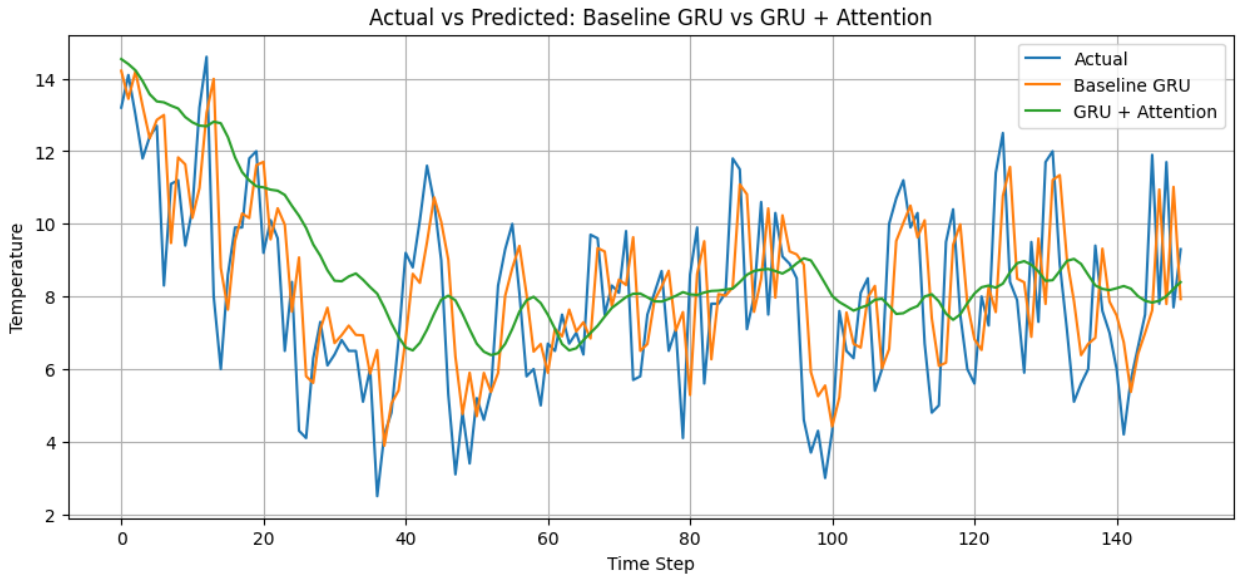

Figure 2.5: *Baseline GRU vs GRU + Attention vs actual values*

Although the attention mechanism can enhance the interpretability of the model, in some cases it does not improve the prediction accuracy. The predicted results generated by the attention model are smoother, and this may lead to underfitting and reduce its ability to capture short-term fluctuations.

2.5.4. Final Model Comparison

Table 2.3: *Final model configurations and comparison*

Model	Sequence_Length	Key_Params	Best_Val_Loss	Test_MSE	Test_MAE
Best GRU	21	units=32, layers=1	0.003293	4.843241	1.704301
Best LSTM	21	units=64, layers=1	0.003362	4.832592	1.714430
Best CNN	12	filters=32, kernel_size=3	0.003397	4.918391	1.727774

LSTM achieves a slightly lower test MSE, GRU is selected as the best model due to its better validation performance and simpler architecture, providing a better balance between accuracy and efficiency.

2.6. Discussion

2.6.1. Why GRU Performed Best

GRU achieves the best performance in this task due to its balance between model complexity and efficiency. Compared to LSTM, GRU has fewer parameters, which reduces the risk of overfitting, especially given the relatively small and smooth dataset. At the same time, it is still capable of capturing temporal dependencies effectively, making it well suited for this forecasting problem.

2.6.2. Why Attention Performed Worse

In this instance, our attention mechanism has not led to improvement. There might be more than one reason for that: Firstly, the prediction tasks are very easy, while temperature series are rather smooth without complicated long-term correlations. Besides, we have very short inputs, so we cannot really gain much from giving different levels of attention to each time step. Finally, the attention mechanism increases model complexity, which may lead to overfitting or unstable training.

2.6.3. CNN vs RNN Interpretation

The performance of the CNN model is competitive but slightly lower than that of GRU. Because by using a convolution operation, CNN has a certain ability to deal with local temporal relationships. However, there is no explicit description about sequence dependency over time here. In comparison, RNN-based models like GRU are intended to handle sequences across time. Therefore, they have better adaptability towards solving problems related to prediction.

2.7. Limitations and Future Work

The above study also has certain limitations. Firstly, for this project, our model architecture itself isn't too complicated, plus we also only adopted one single variable, "temperature." But actually speaking, regarding weather situations, there will always exist multiple influencing factors at the same time, such as rainfall volume, wind speed, humidity, etc., so just adopting one single variable will limit the performance of our model's prediction capability. Secondly, our whole dataset here is rather simple, whose temperature pattern remains relatively steady and doesn't have very complicated ones. Therefore, it might make our model fail to learn well when training, thus impacting its ability to generalize in more complicated situations. Finally, up till now, what we have done is just confined within a one-step prediction task, i.e., predicting tomorrow's temperature value, but practically speaking, usually we need to carry out multi-step prediction tasks to forecast how the trend will change across several successive days. In future research, more variables (e.g., humidity and wind speed) need to be added so that the models can better conform to reality, which may increase the prediction accuracy too. Meanwhile, transforming the present one-step prediction models into multi-step forms would also increase their practicability. In addition, there is room for exploring some even more sophisticated models, like those built upon the Transformer architecture, etc., so as to find out how they perform here.

2.8. Conclusion

In this project, we compared several kinds of deep learning models regarding the task of predicting time series using temperature datasets. We found out that GRU got the highest score; CNN achieved competitive performance too, although it wasn't so strong as GRU. As for the attention mechanism aspect, it helped us to have better explanations about how our models worked to some degree, yet it failed to increase our accuracy when doing predictions here.

To conclude, all these results suggest that an RNN-based model, especially one using GRU units, would be better for this kind of time-series forecasting problem, since it could discover the underlying temporal correlation of the dataset easily.

OVERALL CONCLUSION

This project explored a complete progression of deep learning architectures across two distinct tasks - image classification on the Flickr8k dataset and time series temperature forecasting on London weather data - providing a comprehensive view of how architectural choices interact with task structure.

MLP Models (Part 1): Both architectures plateaued at approximately 53–54% validation accuracy. Flattening 2D images into 1D vectors destroys all spatial structure, making it impossible to learn features such as edges, textures, or shapes. The results confirm that MLPs are fundamentally ill-suited for image classification regardless of depth or regularisation.

CNN Baseline (Part 1): The baseline CNN improved performance to 59.32% by preserving spatial relationships through convolutional and pooling layers. However, the oversized 4×4 and 6×6 pooling windows discarded spatial detail too aggressively, causing clear overfitting from epoch 3 onward and motivating the optimisation experiments.

CNN Optimisation Experiments (Part 1): Replacing large pooling windows with standard 2×2 MaxPooling and substituting the Flatten layer with Global Average Pooling proved most effective, producing a stable model with a significantly tighter train/val gap. Batch Normalisation introduced instability, while the deeper VGG-style architecture with Dropout confirmed that capacity gains require careful paired regularisation. Data augmentation further improved robustness by preventing the model from memorising exact pixel locations.

Interpretability (Part 1): Grad-CAM confirmed that the optimised CNN attends to semantically meaningful image regions such as animal faces and object contours. Saliency maps provided complementary pixel-level evidence. Both techniques validated that the model had learned relevant visual features rather than background artefacts.

Transfer Learning (Part 1): ResNet50 achieved the strongest classification results across Part 1, demonstrating that pretrained ImageNet features transfer powerfully to the Flickr task. Freezing the base layers while training a custom classification head prevented catastrophic forgetting while efficiently adapting the model to the five target categories.

Sequence Prediction (Part 2): The GRU model achieved the best overall performance (Test MSE ≈ 4.85, MAE ≈ 1.70), outperforming both the 1D CNN and the LSTM. Its simpler gating mechanism reduced overfitting risk on a smooth, low-complexity time series while still capturing temporal dependencies effectively. The CNN performed competitively for local pattern extraction but lacked explicit sequential modelling. The attention-augmented GRU underperformed (Test MSE ≈ 9.41), as the added complexity offered no benefit on short, smooth sequences.

Key Takeaway: For image classification on limited datasets, transfer learning from a large pretrained model substantially outperforms training from scratch, even with careful regularisation and augmentation. For time series forecasting on smooth, univariate data, a compact recurrent model consistently beats more complex alternatives. Across both tasks, the results reinforce a unifying principle: model complexity should be matched to task complexity, and simpler, well-regularised architectures frequently outperform over-engineered ones.